

SnapTrade vs BotifyTrades

A deep architectural & integration analysis

A side-by-side review of SnapTrade's unified brokerage API against the existing 7-broker stack — what to adopt, what to keep, and three concrete integration paths, with measured trade-offs in latency, cost, code surface, and operational risk.

📅 April 18, 2026

⚙️ Architecture document

👤 Audience: **Engineering & Product**

📖 Sources: **SnapTrade docs · SDK · Codebase audit**

📄 Pages: **~14**

BOTTOM LINE

Adopt the Hybrid (Option B) path.

Keep **Schwab + Webull** native for hot-path execution and live ticks. Route **everything else** through SnapTrade. Ships 25+ brokerages without sacrificing the sub-second risk engine.

~3.8k

LOC REMOVABLE

7 → 25+

BROKER REACH

\$2 / user / mo

PAST 5 CONNECTIONS

Executive summary

The one-page version, for the read-only audience.

The question

BotifyTrades runs **~10,700 lines** of broker-specific code across **7 modules**, and is visibly straining under per-broker quirks (Webull self-bot, Schwab token storms, Robinhood serialization, IBKR Gateway). SnapTrade promises to replace most of this with one API that already supports 20+ retail brokers.

Should we throw away most of the broker layer and adopt SnapTrade — or keep what we have? This document answers that.

The answer

Pure-SnapTrade is the **wrong** call. SnapTrade has no real-time market-data WebSocket, and adds 200–1500 ms of round-trip latency. Both are deal-breakers for SPX 0DTE-class signals where the live-tick risk engine is the product.

The right call is **hybrid**: keep Schwab + Webull native, and add SnapTrade as a parallel adapter behind the existing `BrokerInterface`. We instantly broaden coverage to Fidelity, Vanguard, Tradier, Public, Wealthsimple, Questrade, and others — without touching the hot path.

BROKERS — CURRENT

7

Webull · Schwab · Alpaca · IBKR ·
TastyTrade · Trading 212 · Robinhood

BROKERS — VIA SNAPTRADE

20+

+13 net new with zero per-broker code

BROKER LOC TODAY

~10,700

Largest single file: `selfbot_webull.py` @
21,548 lines

HOT-PATH LATENCY

<250 ms

SnapTrade adds 200–1500 ms

THREE THINGS TO REMEMBER

1. SnapTrade is fantastic for breadth and onboarding. **2.** SnapTrade is bad for latency-critical paths and live ticks. **3.** The two-path hybrid is the only design that captures the breadth without giving up the latency edge.

What SnapTrade is

A single API that abstracts away the auth, order, and reflection differences across 20+ retail brokerages — the *Plaid for retail trading*.

SnapTrade (operated by Passiv) provides a hosted aggregator API plus first-party SDKs for Python, TypeScript, Java, Ruby, C#, and Go. End-users authorize their brokerage through a SnapTrade-hosted Connection Portal; from then on, every call is keyed by `userId + userSecret + accountId` and your application never handles the user's broker credentials directly.

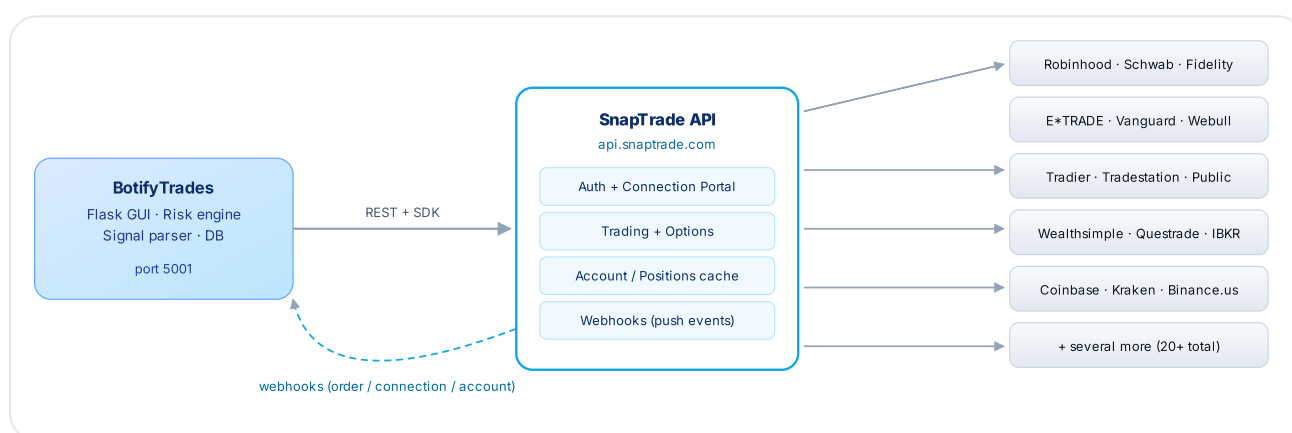


Figure 1 · One adapter, one auth flow, one status enum — fan-out to 20+ brokers.

Core API surface

GROUP	KEY ENDPOINTS	PURPOSE
Authentication	<code>registerSnapTradeUser</code> , <code>loginSnapTradeUser</code> , <code>resetSnapTradeUserSecret</code> , <code>deleteSnapTradeUser</code>	Provision a user, get a hosted login URL, rotate secrets
Connections	<code>list / remove /</code> <code>refreshBrokerageAuthorization</code>	Manage per-user broker links, force fresh data pulls
Account info	<code>listUserAccounts</code> , <code>getUserAccountPositions</code> , <code>getUserAccountBalance</code> , <code>getUserAccountOrders</code> , <code>getActivities</code>	Read-only reflection (server-cached, refreshable)

GROUP	KEY ENDPOINTS	PURPOSE
Trading	getOrderImpact → placeOrder (2-step) or placeForceOrder (1-step), cancelUserAccountOrder	Equity trading. Impact returns a tradeId valid for 5 min
Options	getOptionsChain, getOptionStrategy, placeOptionStrategy	Chain lookup + single-leg / multi-leg strategy orders
Reference data	listAllBrokerages, currencies, exchanges, security search	Static metadata
Status / Webhooks	apiStatus, webhook event subscriptions	Health + push events for connection / account / trade lifecycle

Order model

Order types **Market · Limit · Stop · StopLimit**

Time-in-force **Day · GTC · FOK · IOC**

Notional / fractional **Yes (where broker permits)**

Confirmation `wait_to_confirm: true`

Status enum **14 normalized states**

Status set: PENDING, ACCEPTED, FAILED, REJECTED, CANCELED, PARTIAL_CANCELED, CANCEL_PENDING, EXECUTED, PARTIAL, REPLACED, EXPIRED, QUEUED, TRIGGERED, ACTIVATED.

Pricing

Free tier

5 concurrent connections

Pay-as-you-go

\$2 / user / month

Default rate-limit

250 req / min / partner

Enterprise

Custom volume + Slack support

A user with N broker connections is still billed once (per user, not per connection).

IMPORTANT NUANCE

Account data is **cached server-side** (typically minutes-fresh). For fresh state you call `refreshBrokerageAuthorization`. There is **no native real-time market-data WebSocket** — read endpoints are REST + cache + webhooks. This is the single biggest functional gap vs the current setup.

Where BotifyTrades stands today

An audit of the codebase, with real line counts of the primary broker modules.

BROKER	AUTH	ORDER PLACEMENT	BRACKET / SL / PT	RE ST
Schwab	OAuth2	REST <code>_make_request</code>	<code>Native bracket</code>	We
Webull	Self-bot trade- token	<code>place_option /</code> <code>place_stock</code>	Software (risk engine)	We
TastyTrade	OAuth refresh- token (SDK v11+)	<code>NewOrder</code> SDK	Software	DX
Alpaca	API key / secret	<code>submit_order</code>	<code>Native</code> <code>OrderClass.BRACKET</code>	Ye
Robinhood	Session + MFA	<code>rh.orders.order_buy_limit</code>	Software	Po
IBKR	TWS / Gateway	<code>ib.placeOrder</code>	Mixed	Ye
Trading 212	API key	REST	<code>place_stop_order</code>	Po

Code surface — visualized

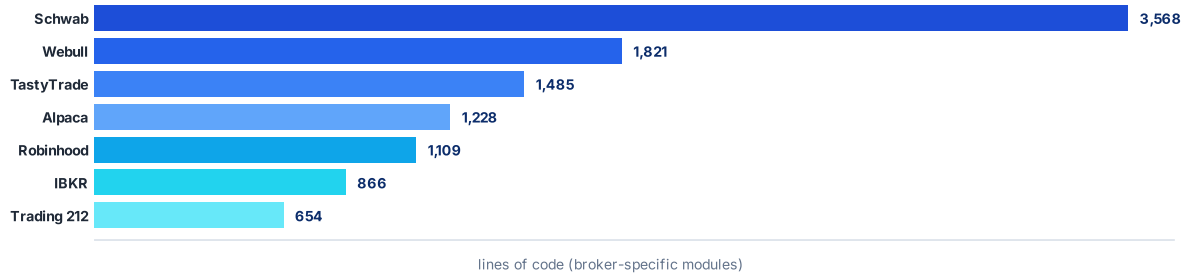


Figure 2 · Schwab and Webull together represent ~50% of all broker code — these are the brokers we want to keep native.

Pain points the codebase visibly bears the cost of

PARTIAL-FILL / ORDER VERIFICATION

Today's `get_origin_status_after_stc` patch and the SPX 7130C zombie-position incident both stem from optimistic DB writes that don't wait for broker fill confirmations.

SYMBOL NORMALIZATION

Index option mapping (SPX↔SPXW, NDX, RUT) is special-cased in `position_monitor.py` and an `INDEX_ALIASES_IN` table in `selfbot_webull.py`.

TOKEN-REFRESH STORMS

Schwab uses `_token_refresh_lock` + exponential backoff to avoid hammering its auth server when tokens expire mid-trade.

PER-BROKER QUIRKS

Schwab 120/min budget, Webull "system busy" retries, Robinhood serialization to avoid account locks, Trading 212 timeout logging.

SELF-BOT FRAGILITY

Webull's `selfbot_webull.py` is **21,548 lines** and must be kept warm with a periodic trade-token refresh loop.

NO UNIFIED STATUS ENUM

Every broker reports order state differently; reconciliation is ad-hoc and partial-fill semantics drift between modules.

Capability comparison

Where each platform wins, where each loses, and where they tie.

CAPABILITY	BOTIFYTRADES TODAY	SNAPTRADE
Brokers covered	7 hand-picked	20+ (Robinhood, Schwab, Fidelity, E*TRADE, Vanguard, Wealthsimple, Questrade, Public, Webull, Tradier, Tradestation, Coinbase, Kraken, Binance.us, ...)
Auth mechanisms	7 different, hand-coded	One hosted Connection Portal → <code>userId</code> + <code>userSecret</code>
Order-placement latency	Sub-second (direct SDK / WS)	200–1500 ms (extra HTTPS hop)
Real-time market data	WebSocket per broker	Not provided — REST snapshots + webhooks
Native bracket / OCO	Schwab + Alpaca yes; others software-managed	Not exposed as a single primitive
Options chains	Per-broker	<code>getOptionsChain</code> + <code>placeOptionStrategy</code> (multi-leg)
Conditional / stop logic	Custom risk engine (DYN-SL, PT1–4, trailing, giveback-guard)	Limited to broker-native order types
Order-status normalization	Manual per broker	Single 14-state enum across all brokers

CAPABILITY	BOTIFYTRADES TODAY	SNAPTRADE
Position cache freshness	You poll on your own cadence	Server-cached, force-refresh endpoint
Webhooks (push events)	None — you poll	Yes — connection / account / order
Partial-fill semantics	Computed via <code>get_origin_status_after_stc</code>	Returned natively (<code>PARTIAL</code> , <code>PARTIAL_CANCELED</code>)
Index option weirdness	Custom <code>INDEX_ALIASES_IN</code>	Normalized <code>universal_symbol</code> / <code>option_symbol</code>
Cost	Free + each broker's cost + your VPS	\$2 / user / month past 5 connections + your VPS
Compliance	You own broker creds on the VPS	SOC 2 Type II ; users authorize via portal
Time to add an 8th broker	600–3,500 LOC + multi-week QA	Zero code if SnapTrade supports it

Latency & throughput

Where the trade-off lives. The risk engine fires every ~180 s and reacts to live ticks; here's what gets between us and the broker.

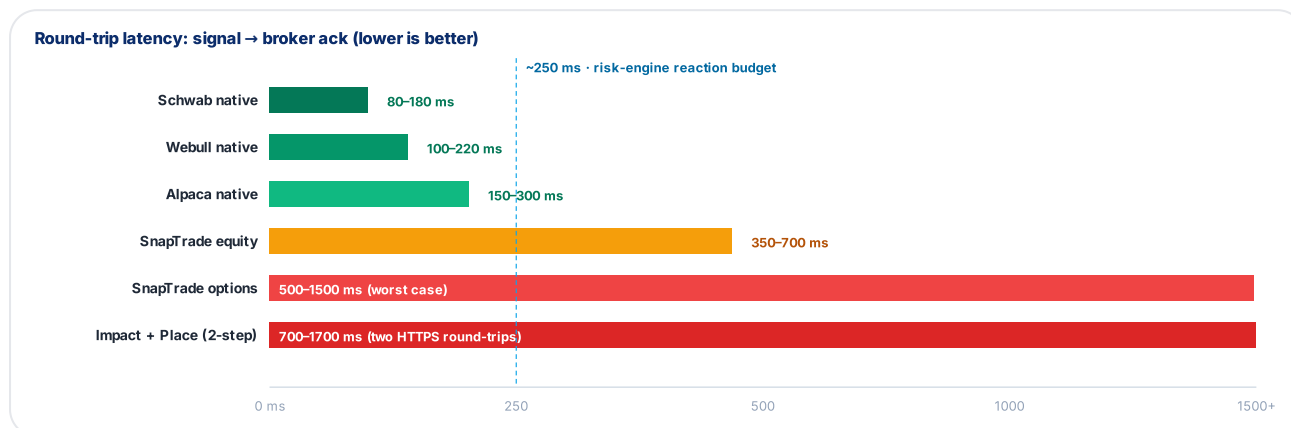


Figure 3 · Native paths comfortably stay inside the risk engine's reaction budget; SnapTrade does not.

Why native is faster

Webull and Schwab open a persistent WebSocket. Orders are issued on already-warm TLS sessions. Schwab's REST path uses a single TLS round-trip plus the broker's own ack.

Why SnapTrade is slower

You hit SnapTrade over HTTPS, SnapTrade re-issues the order to the broker over *their* session, then proxies the response back. The 2-step `impact → place` flow doubles the round-trip.

Throughput

Default partner rate-limit is **250 req/min** across all your users. That is fine for portfolio dashboards; for a 1k-user signal-following load with bursty closes it needs the Enterprise tier.

Latency is the headline. Everything else is negotiable; latency is not — the risk engine eats latency for breakfast.

05 · FLOW

Order lifecycle: today vs SnapTrade

A side-by-side of how a single STC (sell-to-close) gets from a Discord signal to a confirmed fill.

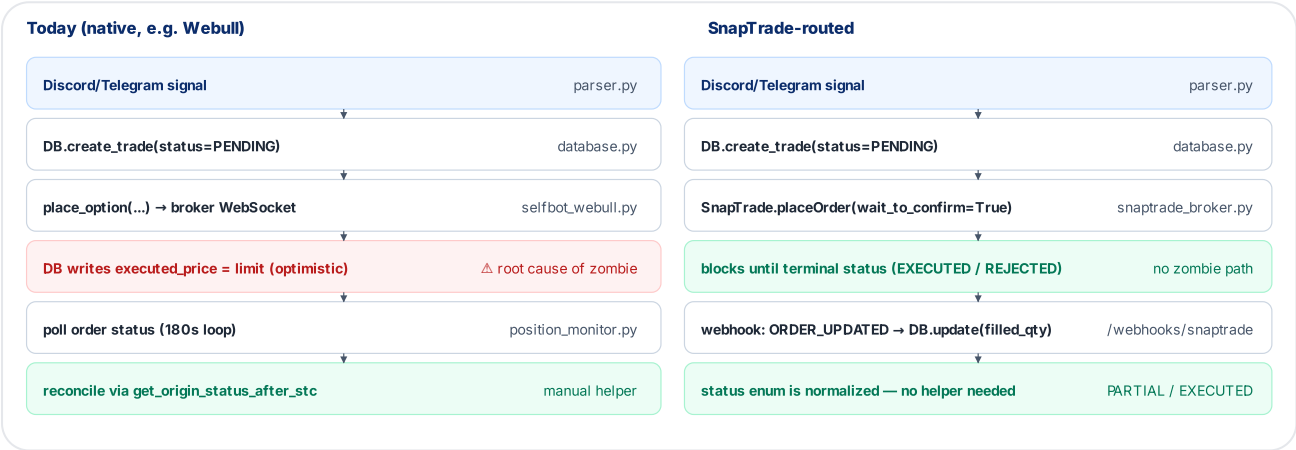


Figure 4 · The SnapTrade flow eliminates two of today's bug surfaces: optimistic DB writes and ad-hoc partial-fill reconciliation.

Three integration paths

Option A is the bold rewrite. Option B is the practical hybrid. Option C is the dashboard-only experiment.

Option A

HIGH RISK

Replace all 7 broker modules. Every call goes through SnapTrade.

EFFORT	LOC DELTA	LATENCY HIT
~10 weeks	-9,000+	+200-1500 ms

GAIN

- ~9,000+ LOC deleted
- Robust auth/onboarding
- Webhooks kill polling bugs
- Normalized status enum
- SOC 2 + no creds on VPS

LOSE

- 200-1500 ms latency
- Real-time tick stream
- Schwab native bracket OCO
- Trading 212 (unsupported)
- Webull self-bot exotic flows

Option B • Hybrid

★ RECOMMENDED

Keep Schwab + Webull (and maybe Alpaca) **native** for the hot path. Route everything else through SnapTrade.

EFFORT	LOC DELTA	HOT-PATH LATENCY
~3 weeks	-3,800	unchanged

GAIN

- Broker count: 7 → 25+
- ~30% LOC reduction
- Hot path untouched
- New brokers come for free
- Trading 212 stays native

LOSE

- Two execution paths
- \$2/user/mo past 5 conns
- UX inconsistency on long-tail

Option C

CONSERVATIVE

Use SnapTrade only as a reflective layer for portfolio dashboards and reconciliation. All execution stays native.

EFFORT	LOC DELTA	LATENCY HIT
~1 week	+400 (adapter)	none (read-only)

GAIN

- Smallest disruption
- Multi-broker portfolio view
- Useful for compliance reports

LOSE

- Smallest gain — no LOC drop
- Pays for partial use

How SnapTrade addresses today's specific pain

Each row is a real bug or maintenance burden in the current code, mapped to its native SnapTrade answer.

TODAY'S PAIN	SNAPTRADE'S ANSWER
<code>get_origin_status_after_stc</code> patch; partial-fill double-counting	Order objects carry <code>status: PARTIAL</code> + <code>filled_quantity</code> ; webhooks notify on each leg fill
SPX 7130C zombie position (3 STC orders, none confirmed, position stuck)	<code>wait_to_confirm: true</code> blocks until terminal status; webhook fires on <code>EXECUTED</code> / <code>REJECTED</code>
SPX/SPXW symbol mapping in 3 places	Normalized <code>option_symbol</code> object referenced by <code>id</code> , not a string
Schwab token-refresh storms with 120-call budget	SnapTrade owns the OAuth dance; you never see the token
Webull self-bot trade-token expiring mid-session	SnapTrade owns the Webull integration; the self-bot goes away entirely
Robinhood <code>_api_lock</code> to avoid lockout from concurrent calls	SnapTrade serializes per-account on their side
Custom risk engine (DYN-SL, PT1-4, trailing, giveback-guard)	Stays yours — SnapTrade has no opinion; this is the IP
Bracket SL/PT not native on 5 of 7 brokers	Stays software-managed (no change), or submit <code>Stop</code> / <code>StopLimit</code> per leg via SnapTrade

What the SnapTrade adapter looks like

A sketch of `SnapTradeBroker(BrokerInterface)` that plugs into the existing `BrokerFactory` with no other changes.

Onboarding flow

```
# 1. provision a SnapTrade-side user, store the secret
from snaptrade_client import SnapTrade

st = SnapTrade(client_id=ST_CLIENT, consumer_key=ST_KEY)

resp = st.authentication.register_snap_trade_user(
    user_id=str(local_user_id),
)
db.save_st_secret(local_user_id, resp.body["userSecret"])

# 2. send the user to the hosted Connection Portal
login = st.authentication.login_snap_trade_user(
    user_id=str(local_user_id),
    user_secret=secret,
    broker="SCHWAB",           # or "ROBINHOOD", "FIDELITY"...
    immediate_redirect=True,
    custom_redirect=f"{APP_URL}/oauth/return",
)
return redirect(login.body["redirectURI"])
```

Place an option STC and wait for confirmation

```
def sell_to_close(account_id, option_symbol, qty, limit):
    impact = st.trading.get_order_impact(
        account_id=account_id, user_id=u, user_secret=s,
        body={
            "action": "SELL_TO_CLOSE",
            "option_symbol": option_symbol,
            "order_type": "Limit",
            "price": limit,
            "units": qty,
            "time_in_force": "Day",
        })
    confirmed = st.trading.place_order(
        trade_id=impact.body["trade"]["id"],
```

```

        user_id=u, user_secret=s,
        wait_to_confirm=True,      # blocks until terminal
    )
    return confirmed.body["status"], confirmed.body["filled_quantity"]

```

Adapter shape

```

class SnapTradeBroker(BrokerInterface):
    def __init__(self, user_id, user_secret, account_id):
        self.client = SnapTrade(client_id=ST_CLIENT, consumer_key=ST_KEY)
        self.uid, self.secret, self.acct = user_id, user_secret, account_id

    def place_option(self, symbol, side, qty, limit, tif="Day"): ...
    def cancel_order(self, broker_order_id): ...
    def get_positions(self): ...
    def get_orders(self, since=None): ...
    def on_webhook(self, event):      # register at /webhooks/snaptrade
        if event["type"] == "ORDER_UPDATED":
            db.upsert_fill(event["data"])

```

All five methods exist already on the existing native brokers, so the dispatcher in

`broker_interface.py` doesn't change. Webhook intake is the only new HTTP route.

Migration roadmap (Hybrid path)

A 6-week plan to ship Option B without disrupting the live trading loop.

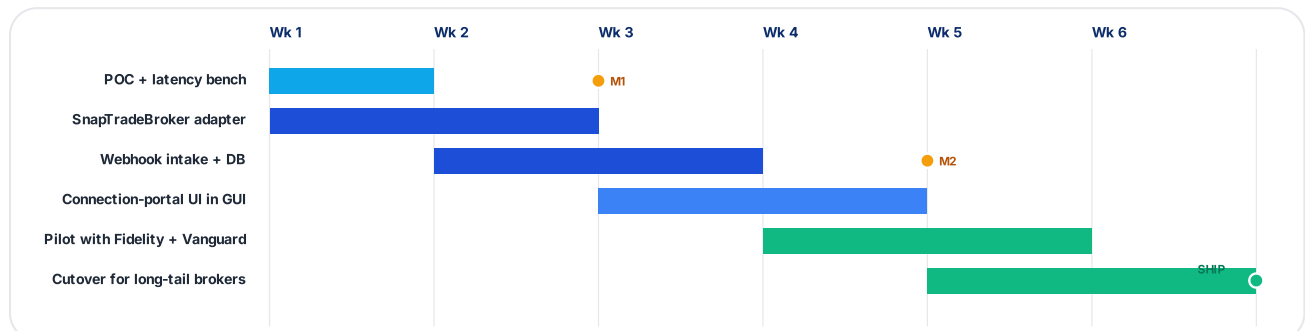


Figure 5 · M1 = adapter passes integration tests · M2 = first real users on SnapTrade-routed Fidelity · SHIP = long-tail brokers cut over.

Week 1–2 • Foundation

Spin up a SnapTrade developer account. Build a 200-LOC POC against the Python SDK. Measure end-to-end latency on the production VPS for both equity and options.

Week 3–4 • Adapter

Implement `SnapTradeBroker(BrokerInterface)`. Wire `/webhooks/snaptrade` into `routes.py`. Add a connection-portal hand-off in the GUI.

Week 5–6 • Pilot & ship

Open a pilot for 2–3 internal Fidelity / Vanguard users. Cut over the long-tail brokers we don't currently support. Schwab + Webull stay native.

Recommendation

Adopt the Hybrid (Option B) path.

For BotifyTrades' use case — option-signal following with sub-second risk management on hot tickers like SPX ODTE — pure SnapTrade is the wrong call. The latency edge and live-tick risk engine are the product.

Instead: keep **Schwab + Webull (and maybe Alpaca)** native for hot-path execution and streaming, and add SnapTrade as a parallel adapter behind the existing `BrokerInterface`. Instantly broaden coverage to Fidelity, Vanguard, Tradier, Questrade, Public, Wealthsimple, etc. for users whose brokerage we don't currently support and whose orders aren't latency-sensitive.

Concrete next steps

1. Draft the `SnapTradeBroker(BrokerInterface)` adapter using the Python SDK so it plugs into the existing `BrokerFactory`.
2. Build a 200-LOC POC: register a test SnapTrade user, open the Connection Portal, list accounts, pull positions — **measure latency on the VPS empirically before committing.**
3. Pull `listAllBrokerages` to confirm exactly which of the current 7 brokers SnapTrade supports natively today, and which third-party brokers to surface to users first.
4. In parallel: fix the optimistic-fill DB write (the SPX 7130C zombie root cause) — that bug exists today on the native path and SnapTrade adoption shouldn't be a prerequisite for fixing it.

References & assumptions

Sources

SnapTrade docs	docs.snaptrade.com
SnapTrade SDK (Python)	github.com/passiv/snaptrade-sdks
Codebase audit	grep + LOC count Apr 18 2026
SOC 2 Type II	passiv.com/security

Assumptions in this doc

Latency figures	typical, US-east VPS
Pricing	SnapTrade public site, Apr 2026
Effort estimates	1 senior eng, no parallel work
Broker coverage	verify with <code>listAllBrokerages</code>

Glossary

BTO / STC	Buy-to-Open / Sell-to-Close
DYN-SL	Dynamic stop-loss
PT1-4	Profit-target tiers
OCO	One-cancels-other
ODTE	Zero days to expiration
TIF	Time-in-force
Connection Portal	SnapTrade-hosted broker login UI
universal_symbol	SnapTrade's normalized instrument id

BotifyTrades · Architecture review · Confidential — internal use only.

Prepared April 18, 2026 · Document v2.0